

Objects and Their Use

Eunbyung Park
Yonsei University



Textbook Chapter 6

Objects and Their Use

In imperative programming, functions are the primary building blocks of program design. In object-oriented programming, objects are the fundamental building blocks in which functions (methods) are a component. We first look at the use of individual software objects in this chapter, and in Chapter 10 look at object design as in object-oriented design.

Disclaimer: some of these slides (where stated) are based on the lecture slides provided by the textbook author, Copyright 2013 John Wiley and Sons. Material added/revised by the course instructor, Copyright 2024, Yonsei University.

These course materials have been produced and published for the purpose of teaching Yonsei University students, so they cannot be used for any other purposes nor be shared with others. The legal responsibility for the violation lies with the actor him/herself.

Overview

- Objects and their use
- Using objects with Python Turtle graphics
- Advanced Python Turtle graphics
- Event-driven programming

Objects: Motivation

The idea of incorporating **objects** into a programming language came out of work in computer simulation. Given the prevalence of objects in the world, it was natural to provide the notion of an object within simulation programs that simulate some aspect of the world.

In the early 1970s, Alan Kay at Xerox PARC (Palo Alto Research Center) fully evolved the notion of object-oriented programming with the development of a programming language called **Smalltalk**.

The language became the inspiration for the development of graphical user interfaces (GUIs)—the primary means of interacting with computer systems today. Before that, all interaction was through typed text.

It was a visit to Xerox PARC by **Steve Jobs** of Apple Computers that led to the development of the first commercially successful GUI-based computer, the Apple Macintosh in 1984.

In this chapter, we look at the creation and use of objects in Python.

Programming Language	Programming Paradigm Supported	
	Procedural	Object-oriented
C (early 1970s)	X	
Smalltalk (1980)		X
C++ (mid 1980s)	X	X
Python (early 1990s)	X	X
Java (1995)		X
Ruby (mid 1990s)	X	X
C # (2000)	X	X

Some Commonly-Used Programming Languages

Objects

- Objects are the fundamental building blocks with object-oriented programming (OOP)
- In Python, **all values** are objects.
 - integers
 - floats
 - strings
 - lists
 - functions
 - ...

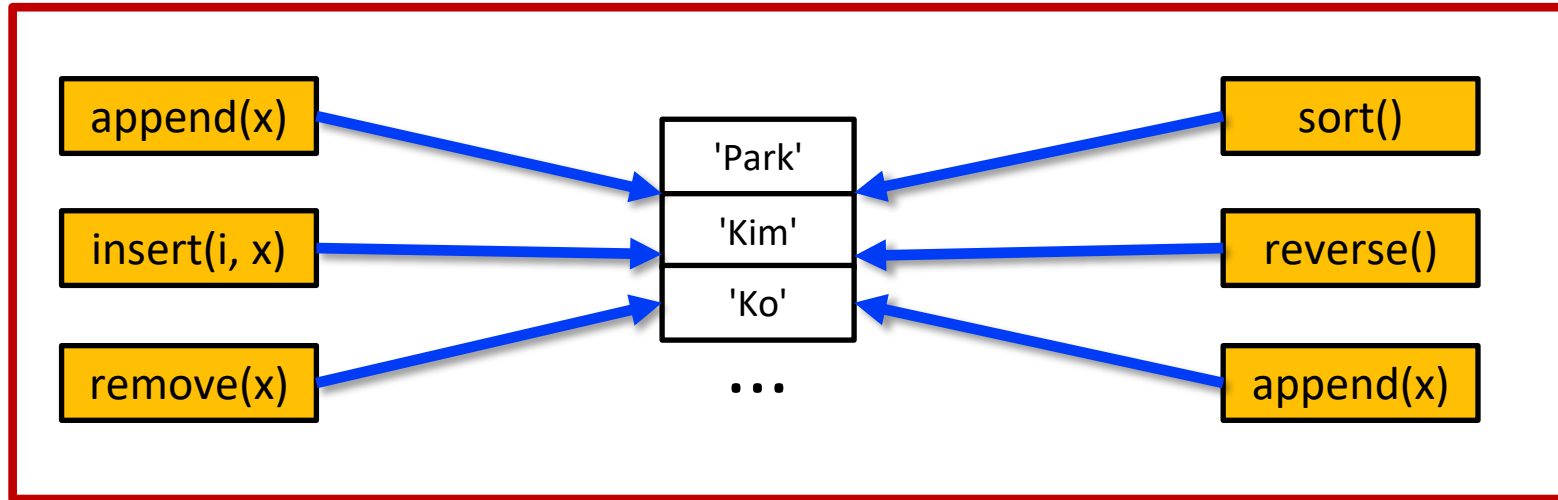
What Is an Object?

An **object** contains:

- **instance variables (data)**
- **functions** (called **methods**) that provide the object's behavior.
- See example on the next slide.

- In this part of the lecture we study how to **use objects**.
- In a latter part of the course, we'll study how to **program classes**, which are the building blocks from which objects are created.

```
name_list = ['Park', 'Kim', 'Ko', ...]
```



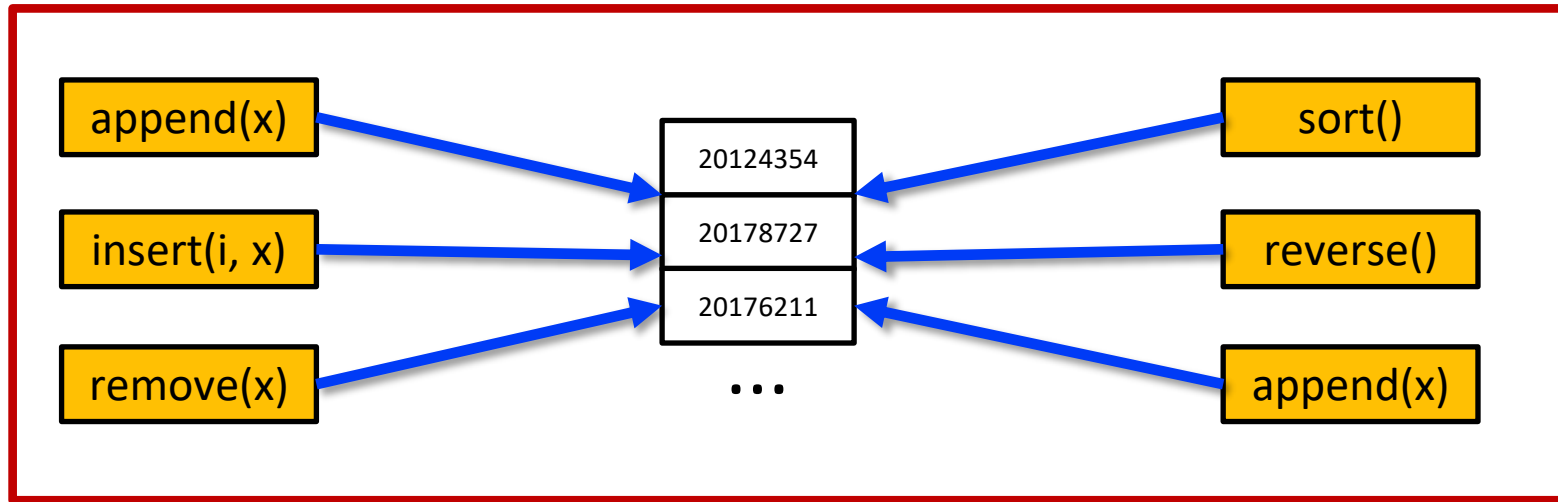
- The `name_list` refers to a **list object**.
 - Every object has a type, in this case type **list**.
- It consists of **data** (strings representing names) and **methods**.
- Methods work on the data of the object.
- Example: `name_list` can be sorted by calling the object's sort method:
`name_list.sort()`

names_list.sort()



- The **period** is called the **dot operator**.
- It selects a member of an object.
- In this case, a method is selected (the sort method).
- **No arguments** are passed to the sort method.
- The object contains **its own data**.
 - The sort method sorts the object's data.
 - Therefore, no arguments need to be provided.
 - The object itself is the argument in this case.

StudentID_list = [20124354, 20178727, 20176211, ...]



- The StudentID_list is another **list object**.
 - It provides the **same methods**.
 - It contains **different data**.
- All list objects provide the **same methods**.
- Different list objects contain **different data**.

Methods Provided by Objects

Every object is of a **specific type**:

- int
- float
- string type (str)
- list
- tuple
- function
- ...

The type of an object determines the methods that the object provides:

- List objects provide append, insert, remove, sort, aso
- Integer objects provide arithmetic operations (+, -, *, ...), relational operations (<, >, ==, ...), aso
- Boolean objects provide logical operations (and, or, not), aso

aso: and so on

Methods versus Functions

- A **method** belongs to an object.
- Methods are called ``on the object'' using the **dot** notation.
 - But infix operators exist, too.
- A method uses the data of the object as input.
 - plus all method arguments

```
l = [1, 3, 5]  
l.append(7)  
l.sort()
```

```
l = 'foo' + 'bar'
```



Infix operator

- A **function** does not belong to any object.
- Functions are called by their name.
 - Don't confuse `math.abs()`, where `math` is the module name.
- Functions require arguments (or global variables) to receive their input.

```
def foo(nums):  
    ...
```

```
foo([1,2,3])
```

Turtle Graphics (1)

- **Turtle graphics** refers to a means of controlling a graphical entity (a “turtle”) in a graphics window with x,y coordinates.
- A turtle can draw lines as it travels, thus creating various graphical designs.
- Turtle graphics was part of a language named **Logo** developed in the 1960s for teaching children how to program.
- Python provides the capability of turtle graphics in the `turtle` Python standard library module.

Turtle Graphics (2)

- There may be more than one turtle on the screen at once.
- Each turtle is represented by a distinct **object**.
- Each object can be individually controlled by the methods provided by turtle objects.
- We introduce turtle graphics here for two reasons—first, to provide a means of better understanding objects in programming, and second, to have some fun!

Creating a Turtle Graphics Window

The first step in the use of turtle graphics is the creation of a turtle graphics window (a **turtle screen**).

```
import turtle
```



```
# set window size:
```

```
turtle.setup(800, 600)
```

```
# get reference to turtle window:
```

```
window = turtle.Screen()
```

```
# set window title bar:
```

```
window.title('My First Turtle Graphics Program')
```

```
window.exitonclick()
```

- To use functions from the **turtle graphics module**, we **import** it.
- Functions are called in the format **turtle.function()**.

module
name

function
name


```
import turtle

# set window size:
turtle.setup(800, 600)

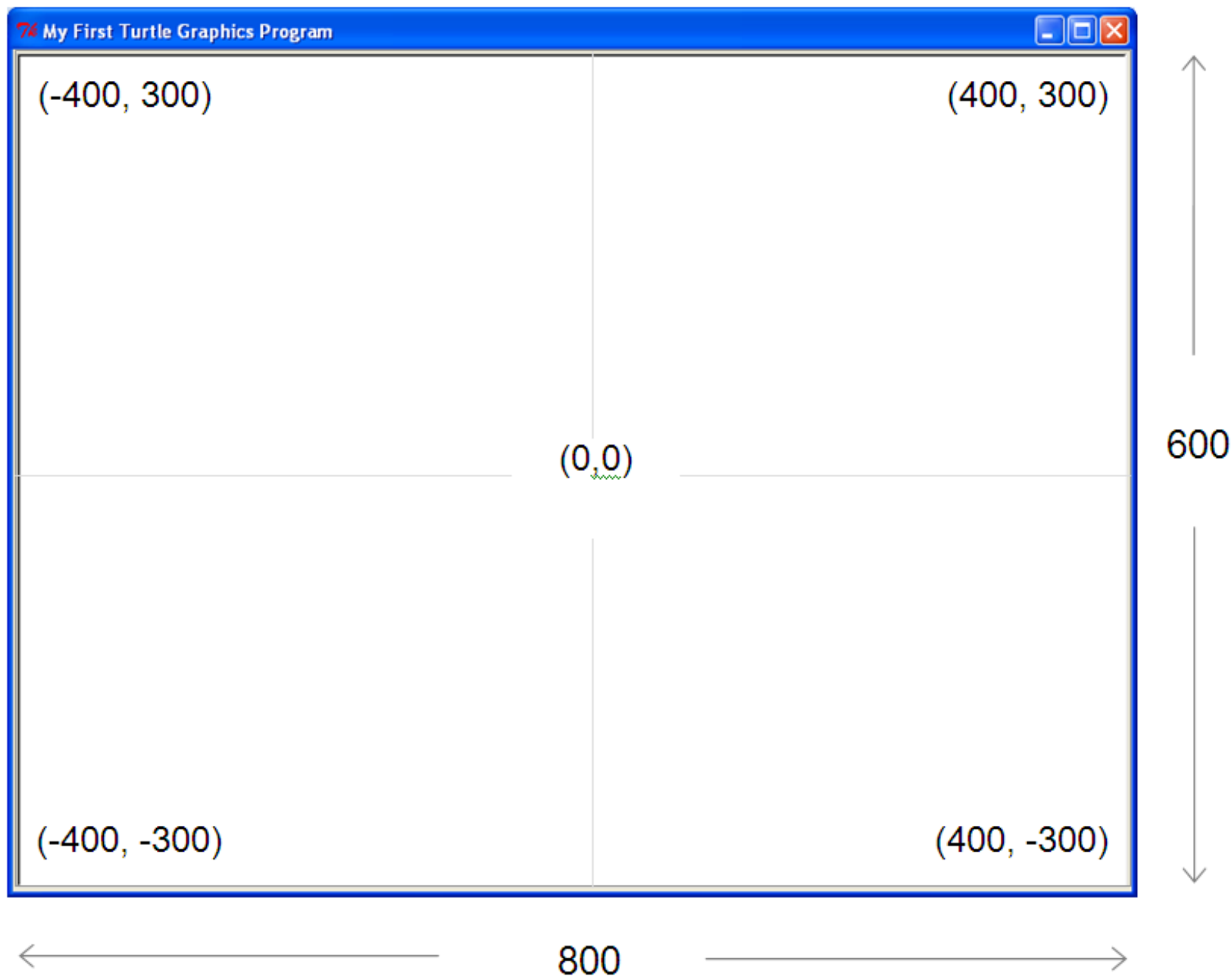
# get reference to turtle window object:
window = turtle.Screen()

# set window title bar:
window.title('My First Turtle Graphics Program')

window.exitonclick()
```



- Calling the **setup()** function of the turtle module creates a **graphics window** of the specified size.
 - The size is specified in units of screen pixels.
 - Width x height.



```
import turtle

# set window size:
turtle.setup(800, 600)

# get reference to turtle window object:
window = turtle.Screen()

# set window title bar:
window.title('My First Turtle Graphics Program')

window.exitonclick()
```



- We call the **Screen()** function of the **turtle module**.
 - Returns the reference to the created screen.
- The screen is an **object**.
- Now our program holds its reference in variable ``window''.

```
import turtle
```

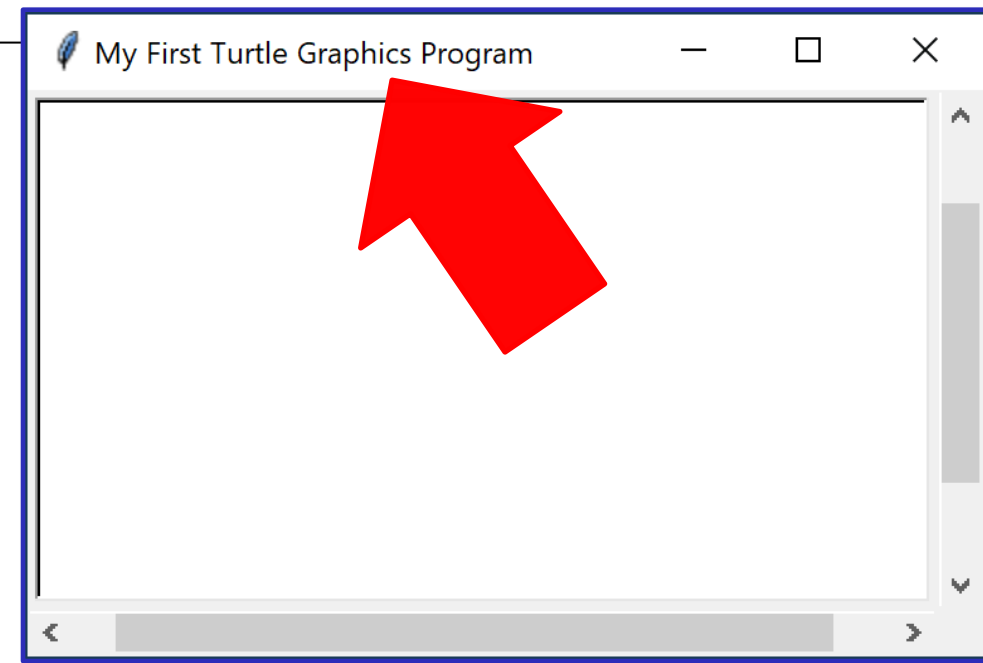
```
# set window size:  
turtle.setup(800, 600)
```

```
# get reference to turtle window:  
window = turtle.Screen()
```

```
# set window title bar:
```

```
window.title('My First Turtle Graphics Program')
```

```
window.exitonclick()
```



- Calling the **title()** method on the **window object** sets the title line of the displayed window.

```
import turtle

# set window size:
turtle.setup(800, 600)

# get reference to turtle window:
window = turtle.Screen()

# set window title bar:
window.title('My First Turtle Graphics Program')

window.exitonclick()
```



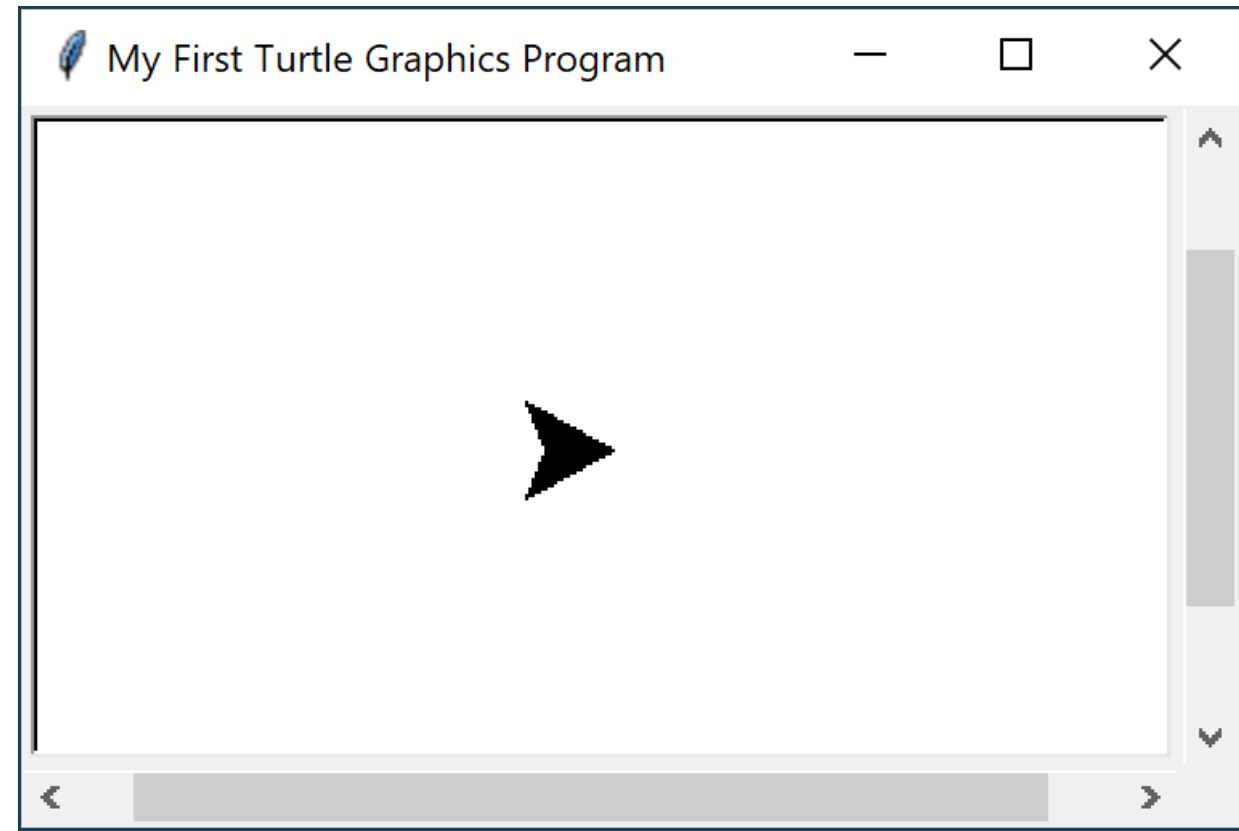
- The window's **exitonclick()** method waits until the user clicks on the window.
 - Otherwise, the window would disappear immediately.
- With the code provided on LearnUs, you find this method call **at the very end of the file!**

The ``Default'' Turtle

- A ``**default'' turtle object** is created when the turtle module's **setup()** function is called.
- The reference to this turtle object can be obtained by:

```
the_turtle = turtle.getturtle()
```

- A call to **getturtle()** causes the default turtle to **appear** in the window.
- The **initial position** of all turtles is the center of the screen at coordinates (0, 0).
- The default turtle shape is an arrowhead.
- The size of the turtle shape has been enlarged for clarity.



Fundamental Turtle Attributes and Behavior

Turtle objects have **three fundamental attributes**:

- *position attributes*
- *heading* (orientation) attributes
- *pen* attributes

Absolute Positioning

- Method **position** returns a turtle's current position.
- Thus, for a newly-created turtle `t.position()` returns the tuple `(0, 0)`.
- A turtle's position can be changed using ***absolute positioning*** by moving the turtle to a specific x,y coordinate by use of method **setposition**.

Drawing a square using absolute positioning

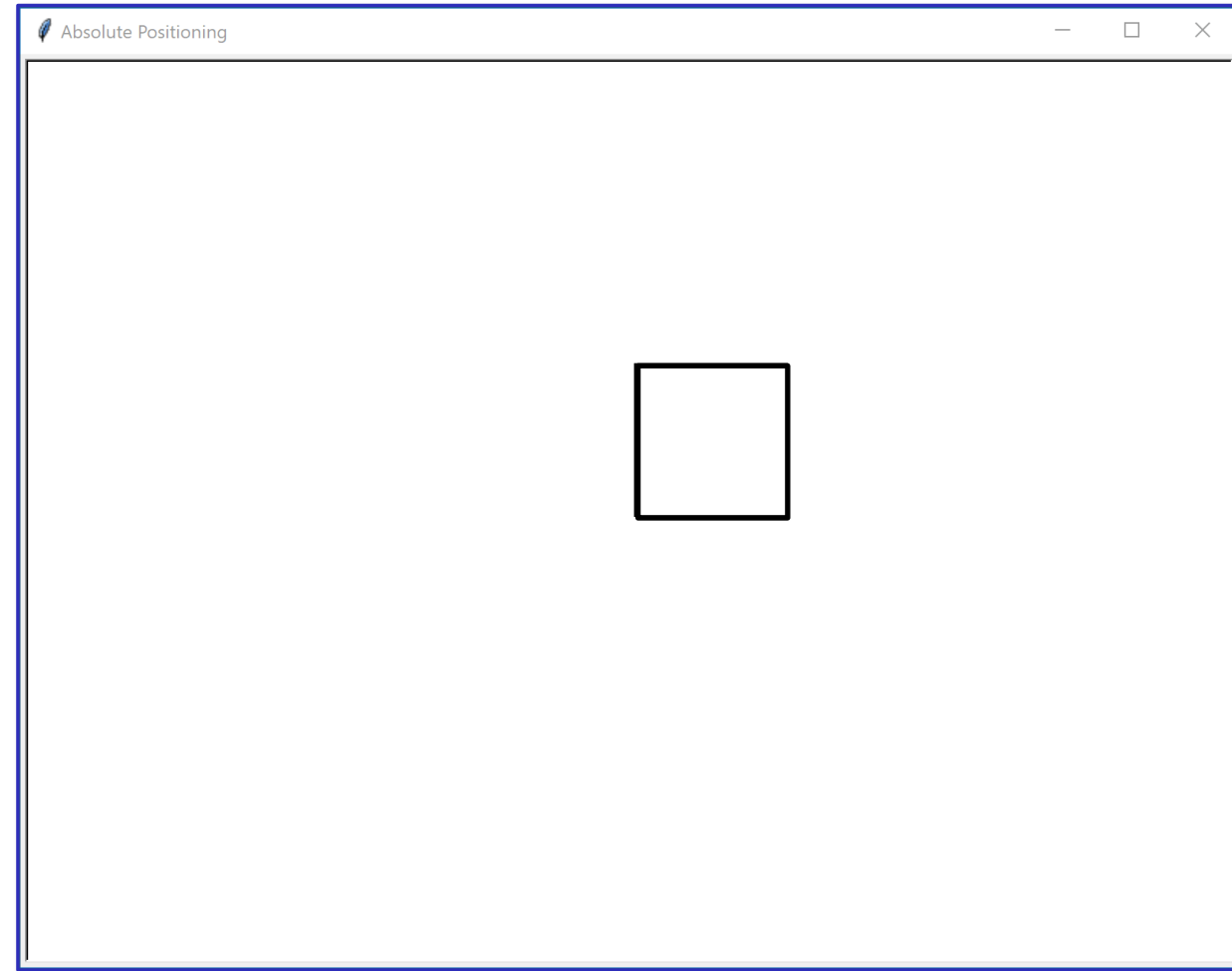
```
# Set window title:  
window = turtle.Screen()  
window.title('Absolute Positioning')
```

```
# Get default turtle and hide:  
the_turtle = turtle.getturtle()  
the_turtle.hideturtle()
```

```
# Draw square using absolute
```

```
# positioning:
```

```
the_turtle.setposition(100, 0)  
the_turtle.setposition(100, 100)  
the_turtle.setposition(0, 100)  
the_turtle.setposition(0, 0)
```



(Pen-width increased to
enhance readability)

Turtle Heading and Relative Positioning

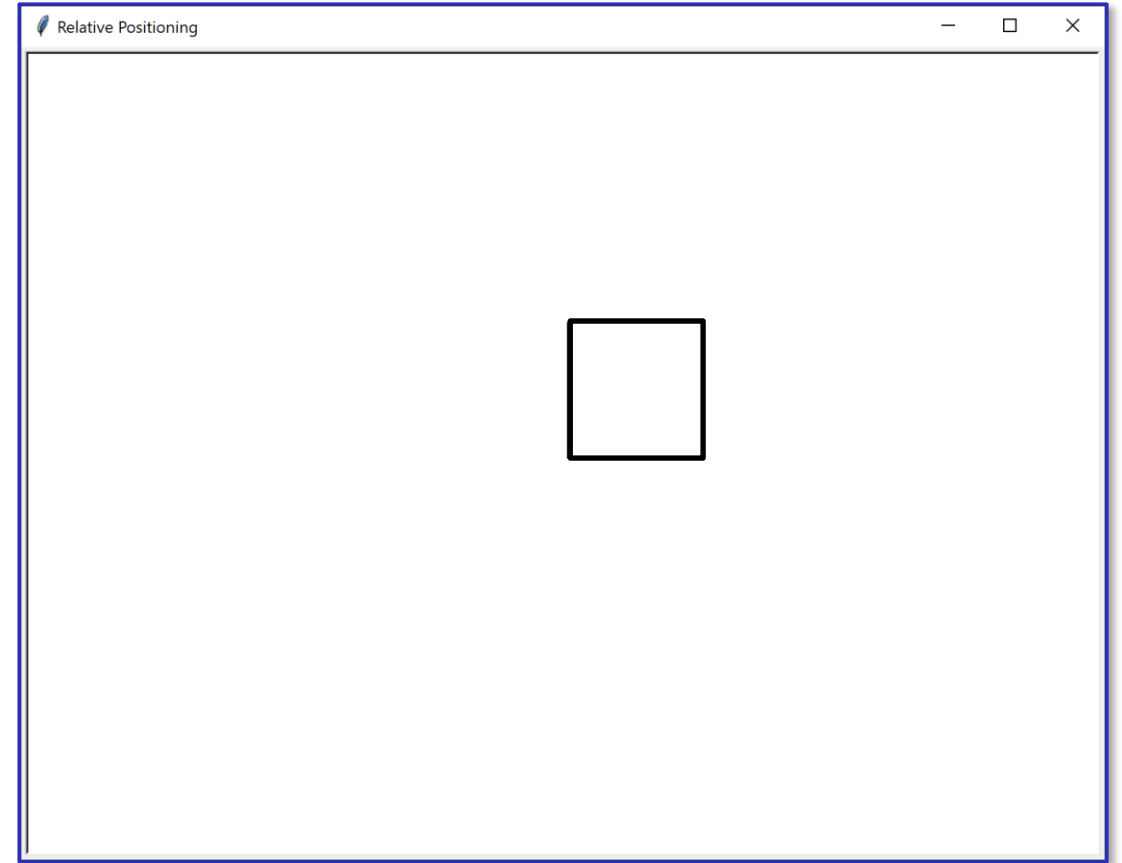
- A turtle's position can be changed through **relative positioning**.
- The turtle will perform a move **relative** to its **current position**.
- Example: `t.forward(100)`
- The direction of the move is determined by the turtle's **heading**.
 - Heading is the attribute that determines the orientation of the turtle.

Drawing a square using relative positioning

```
# Set window title:  
window = turtle.Screen()  
window.title('Relative Positioning')
```

```
# Get default title and hide:  
the_turtle = turtle.getturtle()  
the_turtle.hideturtle()
```

```
# Draw square using relative  
# positioning:  
for move in range(0, 4):  
    the_turtle.forward(100)  
    the_turtle.left(90)
```



(Pen-width increased to
enhance readability)

- Turtles have an initial **heading** of 0 degrees (facing East).
- The turtle is moved **forward** 100 pixels, then **turned left** by 90 degrees.
- Doing this 4x in a loop completes the square.

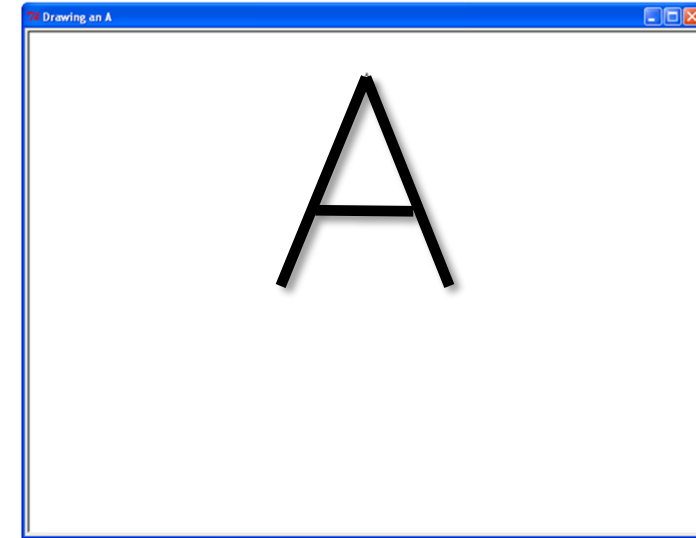
Pen Attributes: Pen Up and Down

- The pen attribute of a turtle object is related to its drawing capabilities.
- The turtle's pen can be “up” or “down,” controlled by methods **penup()** and **pendown()**.
- When the pen attribute value is “up,” the turtle can be moved to another location without lines being drawn.
- This is needed when creating drawings with disconnected parts.

Drawing disconnected lines

```
turtle.setup(800,600)
window = turtle.Screen()
window.title('Drawing an A')

the_turtle = turtle.getturtle()
the_turtle.hideturtle()
the_turtle.penup()
the_turtle.setposition(-100, 0)
the_turtle.pendown()
the_turtle.setposition(0, 250)
the_turtle.setposition(100, 0)
the_turtle.penup()
the_turtle.setposition(-64, 90)
the_turtle.pendown()
the_turtle.setposition(64, 90)
```



(Pen-width increased to enhance readability)

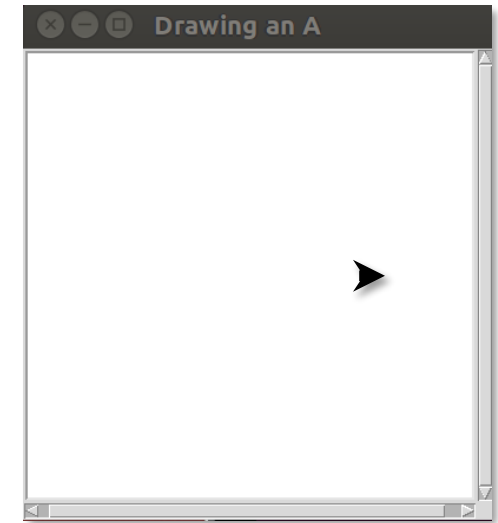
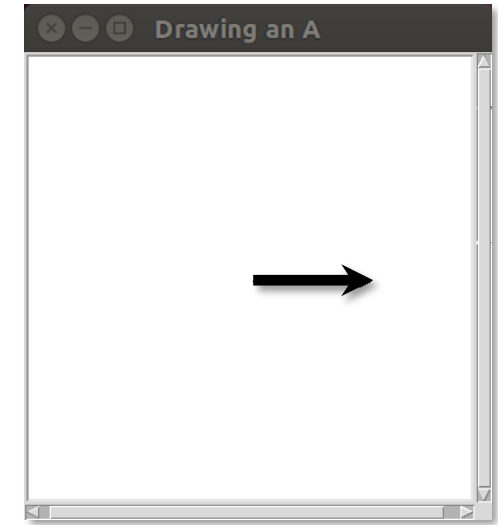
- The **penup** and **pendown** methods are used to allow the turtle to be **repositioned**.

Erasing a turtle's drawing:

```
turtle.setup(400, 400)
window = turtle.Screen()
window.title('Drawing an A')

the_turtle = turtle.getturtle()
the_turtle.setposition(100,0)
input('Press enter to continue')
the_turtle.clear()
input('Press enter to continue')
```

- The **clear()** method erases a turtle's drawing.
- Keeps the turtle's position.
- Note the input() statements to facilitate **debugging!** 😊

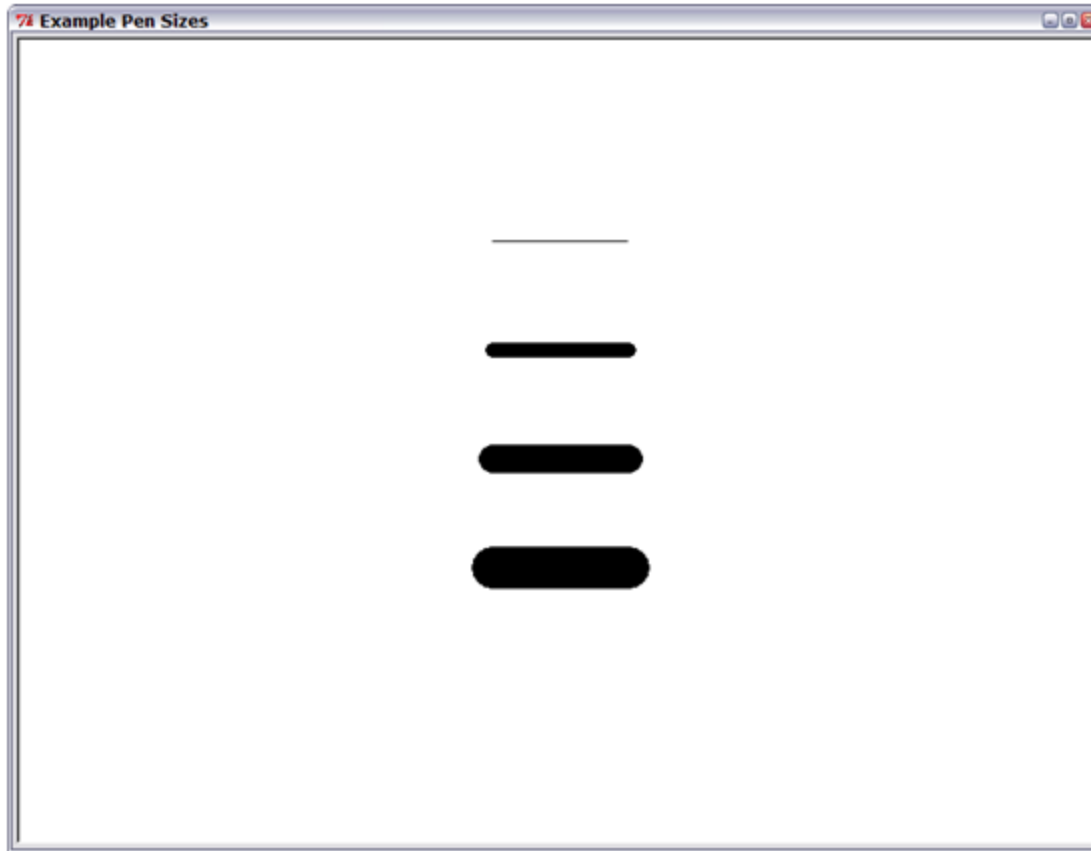


(Pen-width increased to enhance readability)

Pen Attributes: Line Width (1)

- The pen size of a turtle determines the width of the lines drawn when the pen attribute is ``down``.
- The `pensize` method is used to control the pen size.
- Example: `the_turtle.pensize(5)`
- The line width is given in pixels, and is limited only by the size of the turtle screen.
- See examples on next slide.

Pen Attributes: Line Width (2)



`pensize(1)`

`pensize(11)`

`pensize(21)`

`pensize(41)`

Pen Attributes: Pen Color

- The pen color can be selected by use of the `pencolor` method, `the_turtle.pencolor('blue')`.
 - The name of any common color can be used, for example `'white'`, `'red'`, `'blue'`, `'green'`, `'yellow'`, `'gray'`, and `'black'`.
- Colors can also be specified in RGB (red/green/blue) component values. These values can be specified in the range 0–255 if the color mode attribute of the turtle window is set as given below,
 - `turtle.colormode(255)`
 - `the_turtle.pencolor(238, 130, 238) # violet`
- This allows for a full spectrum of colors to be displayed.

Additional Turtle Attributes

- In addition to the fundamental turtle attributes already discussed, **there are other attributes of a turtle that may be controlled.**
- This includes the **size**, **shape**, and **fill color** of the turtle, the turtle's **speed**, and the **tilt** of the turtle.
- See next slides.

Turtle Size

- The turtle size can be controlled through two methods:
 - `resizemode`
 - `turtlesize`
- **First**: set the **resize** attribute of the turtle to **'user'**.
 - Allows the user (programmer) to change the size of the turtle by use of method `turtlesize`.
 - Otherwise, a call to the method will have **no effect**.
- **Second**: set the desired size by providing two stretch factors
 1. in the turtles's orientation, and
 2. perpendicular to the turtle's orientation.

```
# Allow turtle size change by programmer:
```

```
the_turtle.resizemode('user')
```

```
# Change turtle size:
```

```
the_turtle.turtlesize(3, 3) # stretch 3x in both x, y
```

Turtle Shape

- There are a number of ways that a turtle's shape (and fill color) may be defined to something other than the default shape (the arrowhead) and fill color (black).
- A turtle may be assigned one of the following provided shapes: `'arrow'`, `'turtle'`, `'circle'`, `'square'`, `'triangle'`, and `'classic'` (the default arrowhead shape)



- The `shape` and `fill colors` are set by use of the `shape` and `fillcolor` methods.

```
the_turtle.shape('circle')
```

```
the_turtle.fillcolor('white')
```

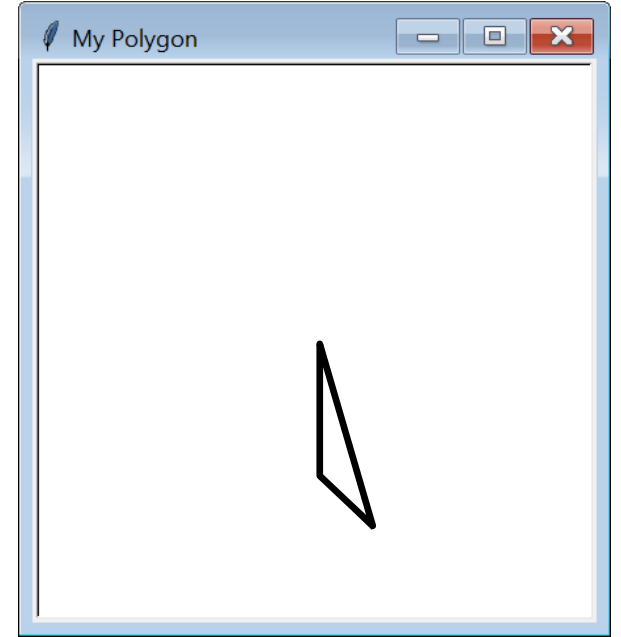
- New shapes may be created and registered with (added to) the turtle screen's *shape dictionary*.
- One way of creating a new shape is by providing a set of coordinates denoting a polygon.

Creating a Polygon Turtle Shape (1)

```
turtle.setup(430,430)
window = turtle.Screen()
window.title('My Polygon')
the_turtle = turtle.getturtle()
```

```
turtle.register_shape('mypolygon',
    ( (0,0), (100,0), (140, 40)) )
```

```
the_turtle.shape('mypolygon')
the_turtle.fillcolor('white')
```



(Pen-width increased to enhance readability)

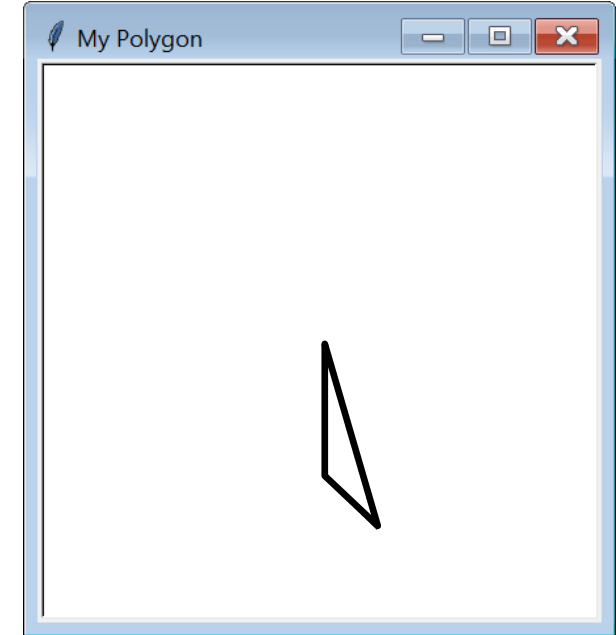
- Method **register_shape()** registers a new turtle shape of the **provided name**.
 - The new shape is provided by a tuple of coordinates (second argument).
- The turtle can be set to the registered shape using the **shape()** method.

Creating a Polygon Turtle Shape (2)

```
turtle.setup(430,430)
window = turtle.Screen()
window.title('My Polygon')
the_turtle = turtle.getturtle()

turtle.register_shape('mypolygon',
    ( (0,0), (100,0), (140, 40)) )

the_turtle.shape('mypolygon')
the_turtle.fillcolor('white')
```



(Pen-width increased to enhance readability)

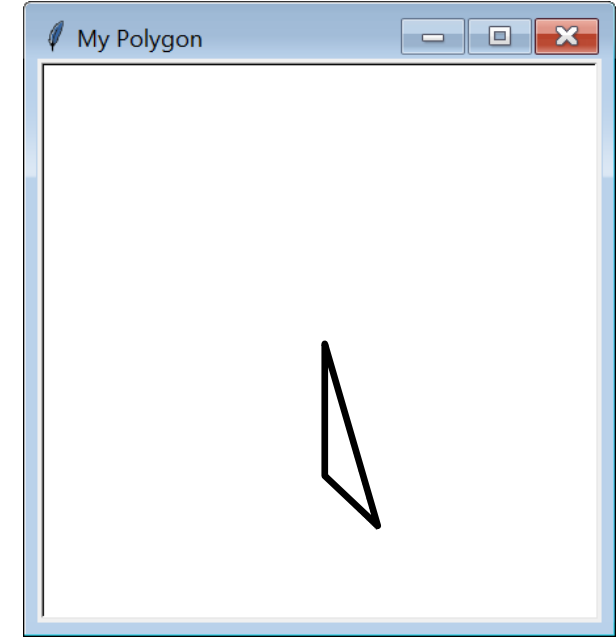
- Method **register_shape()** registers a new turtle shape of the **provided name**.
 - The new shape is provided by a **tuple of coordinates** (second argument).
- The turtle can be set to the registered shape using the **shape()** method.

Creating a Polygon Turtle Shape (3)

```
turtle.setup(430,430)
window = turtle.Screen()
window.title('My Polygon')
the_turtle = turtle.getturtle()

turtle.register_shape('mypolygon',
    ( (0,0), (100,0), (140, 40)) )

the_turtle.shape('mypolygon')
the_turtle.fillcolor('white')
```



(Pen-width increased to enhance readability)

- Method **register_shape()** registers a new turtle shape of the **provided name**.
 - The new shape is provided by a **tuple of coordinates** (second argument).
- The turtle can be set to the registered shape using the **shape()** method.

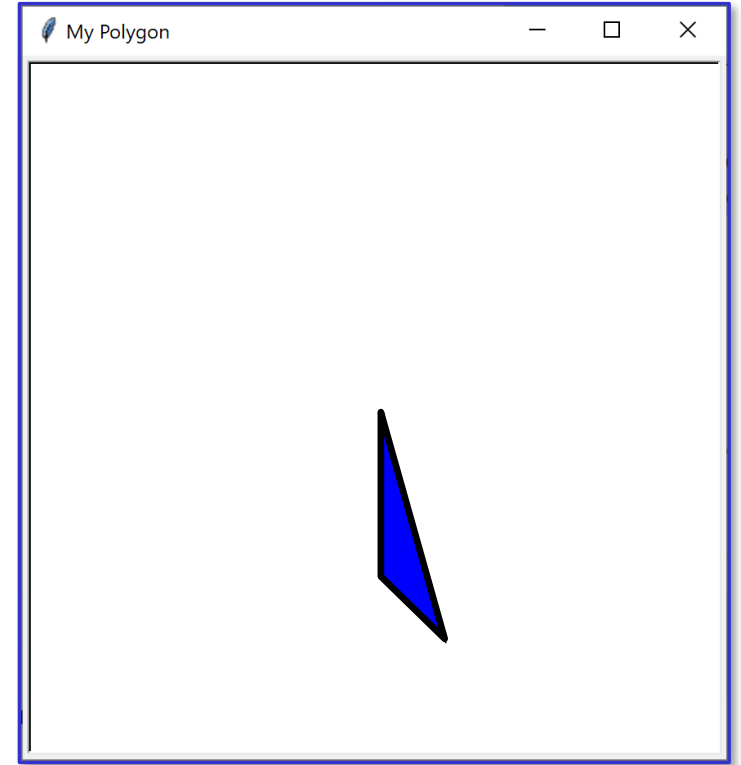
Creating a Polygon Turtle Shape (4)

```
turtle.setup(430,430)
window = turtle.Screen()
window.title('My Polygon')
the_turtle = turtle.getturtle()

turtle.register_shape('mypolygon',
    ( (0,0), (100,0), (140, 40)) )

the_turtle.shape('mypolygon')
the_turtle.fillcolor('blue')
```

- The **fillcolor()** method sets the color of the polygon.
 - Specifying a turtle consisting of multiple polygons is possible, too.



(Pen-width increased to enhance readability)

Stamping a Turtle on the Screen (1)

- A turtle is able to stamp its shape on the screen by use of the `stamp` method, which remains there even after the turtle is repositioned (or relocated).
- That means that we can create all sorts interesting graphic patterns by appropriately repositioning the turtle.

Stamping a Turtle on the Screen (2)

```
turtle.setup(430,430)
```

```
window = turtle.Screen()
```

```
window.title('My stamps')
```

```
the_turtle = turtle.getturtle()
```

```
turtle.register_shape('mypolygon',  
((0,0), (100,0), (140, 40)))
```

```
the_turtle.shape('mypolygon')
```

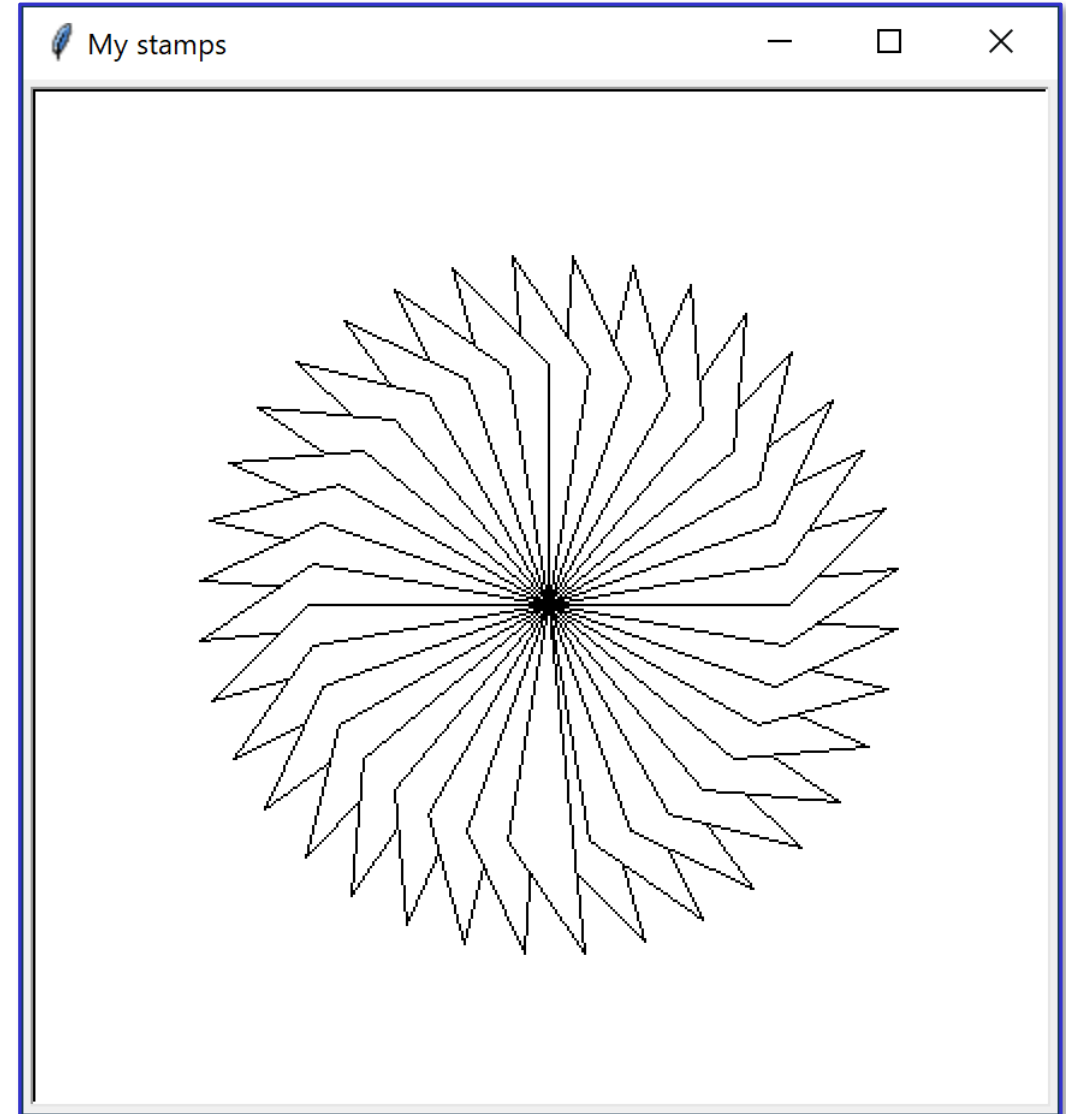
```
the_turtle.fillcolor('white')
```

```
for angle in range(0, 360, 10):
```

```
    the_turtle.setheading(angle)
```

```
    the_turtle.stamp()
```

- The for loop iterates in increments of 10 degrees.
- Method `setheading()` sets the orientation of the turtle.
- Method `stamp()` ``stamps'' the turtle's image on the screen.



Creating a Turtle Shape from an Image

- Another way that a turtle shape can be created is by use of an existing image.
- The image file used must be in the **GIF image format**
 - File extension .gif is commonly used for this format.
- The name of the file is then registered and the shape of the turtle set to the registered name:

```
turtle.register_shape('image1.gif')
```

```
the_turtle.shape('image1.gif')
```

Creating Multiple Turtles

- To create a **new turtle object**, the **Turtle()** function is used.
- The created turtle is **different** from the **``default" turtle** that we've used until now!

```
t1 = turtle.Turtle()  
t2 = turtle.Turtle()  
t3 = turtle.Turtle()
```

- Turtle objects can be stored in a list:

```
turtles = []  
turtles.append(t1)  
turtles.append(t2)  
turtles.append(turtle.Turtle())
```

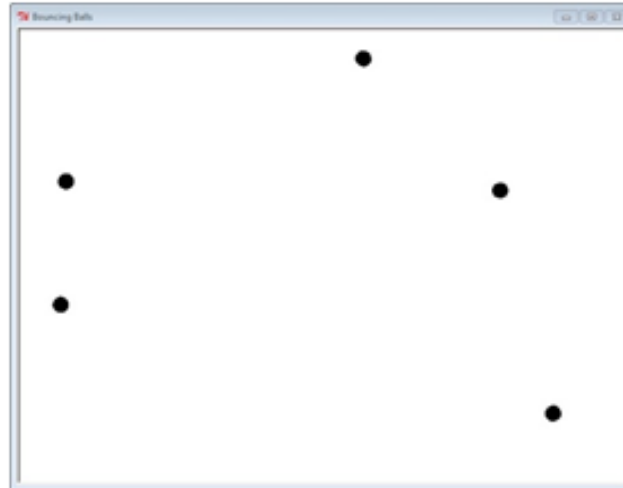
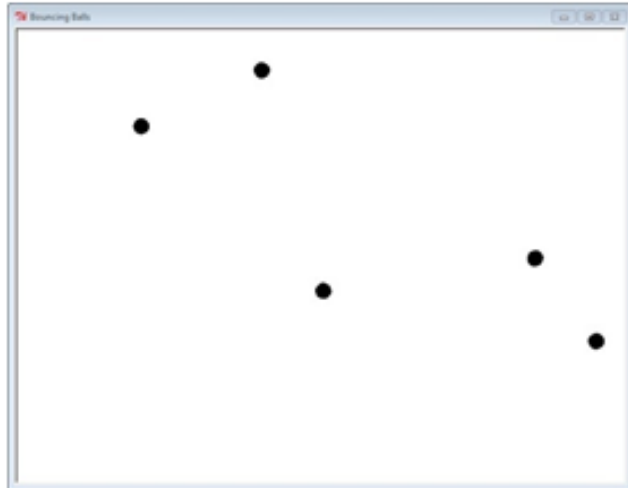
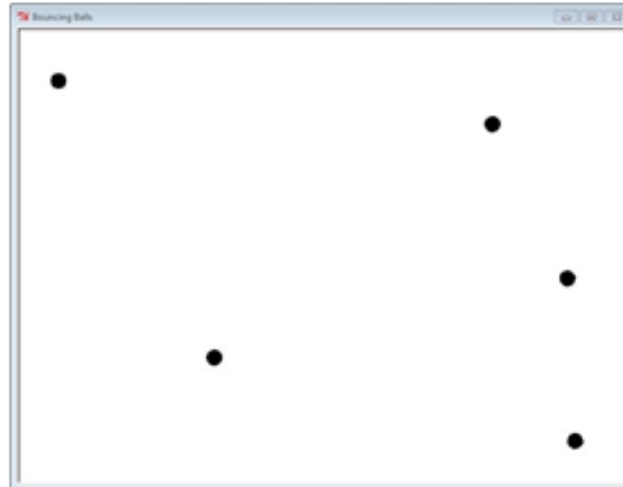
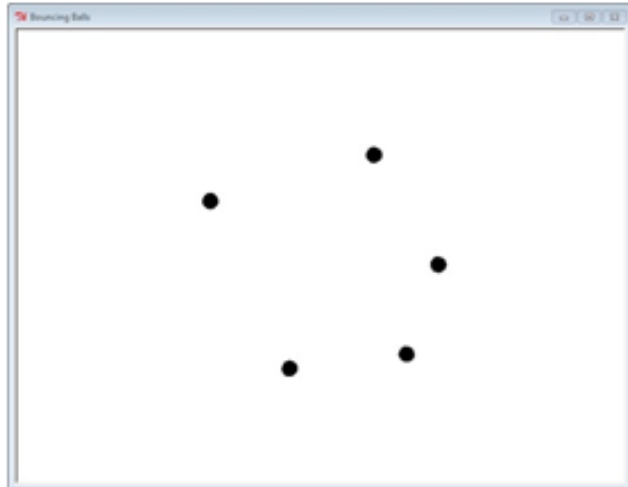
Let's Apply It

Bouncing Balls Program

- The following program displays one or more bouncing balls within a turtle screen window.
- This program utilizes the following programming features.
 - turtle module
 - time module

Program Execution ...

This program simulates one or more bouncing balls on a turtle screen.
Enter number of seconds to run: 30
Enter number of balls in simulation: 5



```

1 # Bouncing Balls Simulation Program
2
3 import turtle
4 import random
5 import time
6
7 def atLeftEdge(ball, screen_width):
8     if ball.xcor() < -screen_width / 2:
9         return True
10    else:
11        return False
12
13 def atRightEdge(ball, screen_width):
14     if ball.xcor() > screen_width / 2:
15         return True
16    else:
17        return False
18
19 def atTopEdge(ball, screen_height):
20     if ball.ycor() > screen_height / 2:
21         return True
22    else:
23        return False
24
25 def atBottomEdge(ball, screen_height):
26     if ball.ycor() < -screen_height / 2:
27         return True
28    else:
29        return False
30
31 def bounceBall(ball, new_direction):
32     if new_direction == 'left' or new_direction == 'right':
33         new_heading = 180 - ball.heading()
34     elif new_direction == 'down' or new_direction == 'up':
35         new_heading = 360 - ball.heading()
36
37     return new_heading
38
39 def createBalls(num_balls):
40     balls = []
41     for k in range(0, num_balls):
42         new_ball = turtle.Turtle()
43         new_ball.shape('circle')
44         new_ball.fillcolor('white')
45         new_ball.speed(0)
46         new_ball.penup()
47         new_ball.setheading(random.randint(1, 359))
48         balls.append(new_ball)
49
50     return balls
51

```

In addition to the turtle graphics module, the time and random Python standard library modules are imported to allow control of how long the simulation is executed, and to generate the random motion of the bouncing balls.

Functions `atLeftEdge`, `atRightEdge`, `atTopEdge`, and `atBottomEdge` (**lines 7-37**) are used to determine when a ball should be bounced off of a wall. Function `bounceBall` is called to do the bouncing.

Function `createBalls` (**lines 39–50**) initializes an empty list named `balls` and creates the requested number of balls one-by-one, each appended to the list. Each ball is created with shape 'circle', fill color of 'white', speed of 0 (fastest speed), and pen attribute 'up'. In addition, the initial heading of each ball turtle is set to a random angle between 1 and 359 (**line 47**).


```

52 # ---- main
53 # program greeting
54 print('This program simulates one or more bouncing balls on a turtle
55 screen.')
56
57 # init screen size
58 screen_width = 800
59 screen_height = 600
60 turtle.setup(screen_width, screen_height)
61
62 # create turtle window
63 window = turtle.Screen()
64 window.title('Bouncing Balls')
65
66 # prompt user for execution time and number of balls
67 num_seconds = int(input('Enter number of seconds to run: '))
68 num_balls = int(input('Enter number of balls in simulation: '))
69
70 # create balls
71 balls = createBalls(num_balls)
72
73 # set start time
74 start_time = time.time()
75
76 # begin simulation
77 terminate = False
78
79 while not terminate:
80     for k in range(0, len(balls)):
81         balls[k].forward(15)
82
83         if atLeftEdge(balls[k], screen_width):
84             balls[k].setheading(bounceBall(balls[k], 'right'))
85         elif atRightEdge(balls[k], screen_width):
86             balls[k].setheading(bounceBall(balls[k], 'left'))
87         elif atTopEdge(balls[k], screen_height):
88             balls[k].setheading(bounceBall(balls[k], 'down'))
89         elif atBottomEdge(balls[k], screen_height):
90             balls[k].setheading(bounceBall(balls[k], 'up'))
91
92         if time.time() - start_time > num_seconds:
93             terminate = True
94
95 # exit on close window
96 turtle.exitonclick()

```

The while loop on **line 79** begins the simulation. The loop iterates as long as Boolean variable `terminate` is not `True` (initialized to `False` on **line 77**). The for loop within this loop at **line 80** moves each of the specified number of balls a small distance until reaching one of the four edges of the window (left, right, top, or bottom), using Boolean functions `atLeftEdge`, `atRightEdge`, `atTopEdge`, and `atBottomEdge`.

Function `bounceBall` is called to bounce the ball in the opposite direction heading, and returns the new heading of the ball, passed as the argument to that ball's `setheading` method. Finally, on **line 92** a check is made to determine whether the user-requested simulation time has been exceeded. If so, the Boolean variable `terminate` is set to `True`, and the program terminates. Because of the call to `exitonclick()` on **line 96**, the program will properly shut down when the close button of the turtle window is clicked on.

Self-Test Questions

1. A turtle screen is an 800-pixel wide by 600-pixel high graphics window. (True/False).
2. The three main attributes of a turtle object are _____, _____, and _____.
3. A turtle can be moved using either _____ or _____ positioning.
4. A turtle can only draw lines when it is not hidden. (True/False)
5. A turtle shape is limited to an arrow, turtle, circle, square, triangle, or classic (default) shape. (True/False)
6. What attribute of a turtle determines the size of the lines it will draw?
 - (a) Pen size
 - (b) Turtle size
7. A turtle can draw in one of seven colors. (True/False)
8. A turtle can leave an imprint of its shape on the screen by use of the _____ method.
9. In order to create a new turtle object, the _____ method is called.

ANSWERS:

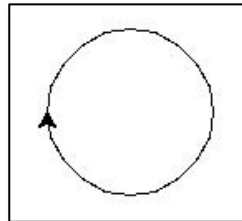
Advanced Python Turtle Graphics

- Basic shapes & iteration
- Controlling animation speed and screen refresh rate
- Filling of shapes

Drawing Circles

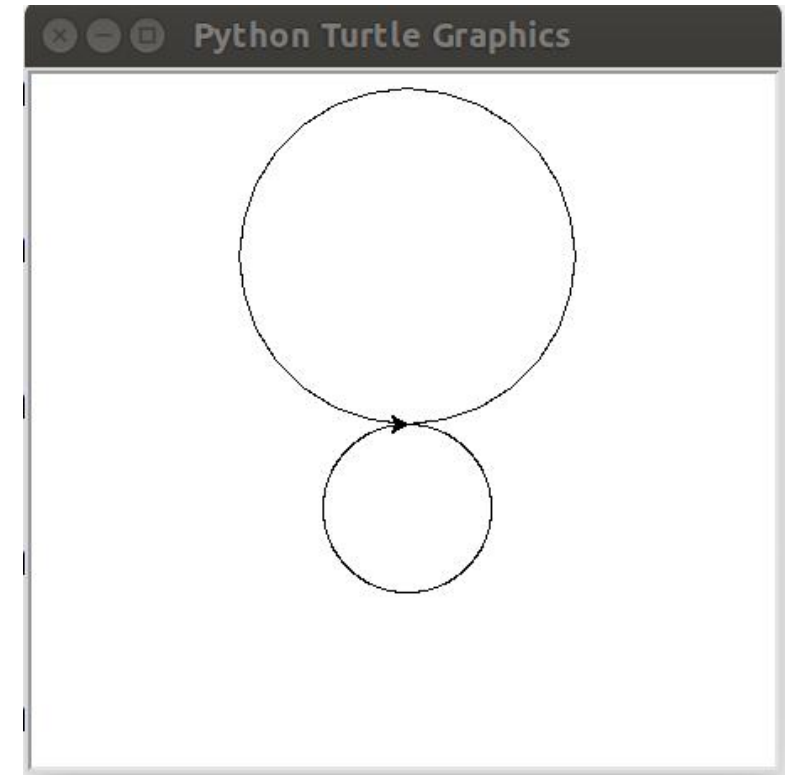
- The `circle()` function draws a circle of the specified radius.
- The circle is drawn either in clockwise direction or counter-clock-wise direction.
- If the radius is **positive**, the circle is drawn **counter-clock-wise**.
- If the radius is **negative**, the circle is drawn **clock-wise**.
- The circle is drawn starting with the current heading (direction) of the turtle:

```
t.left(90)  
t.circle(-50)
```



```
t.circle(100)
```

```
t.circle(-50)
```



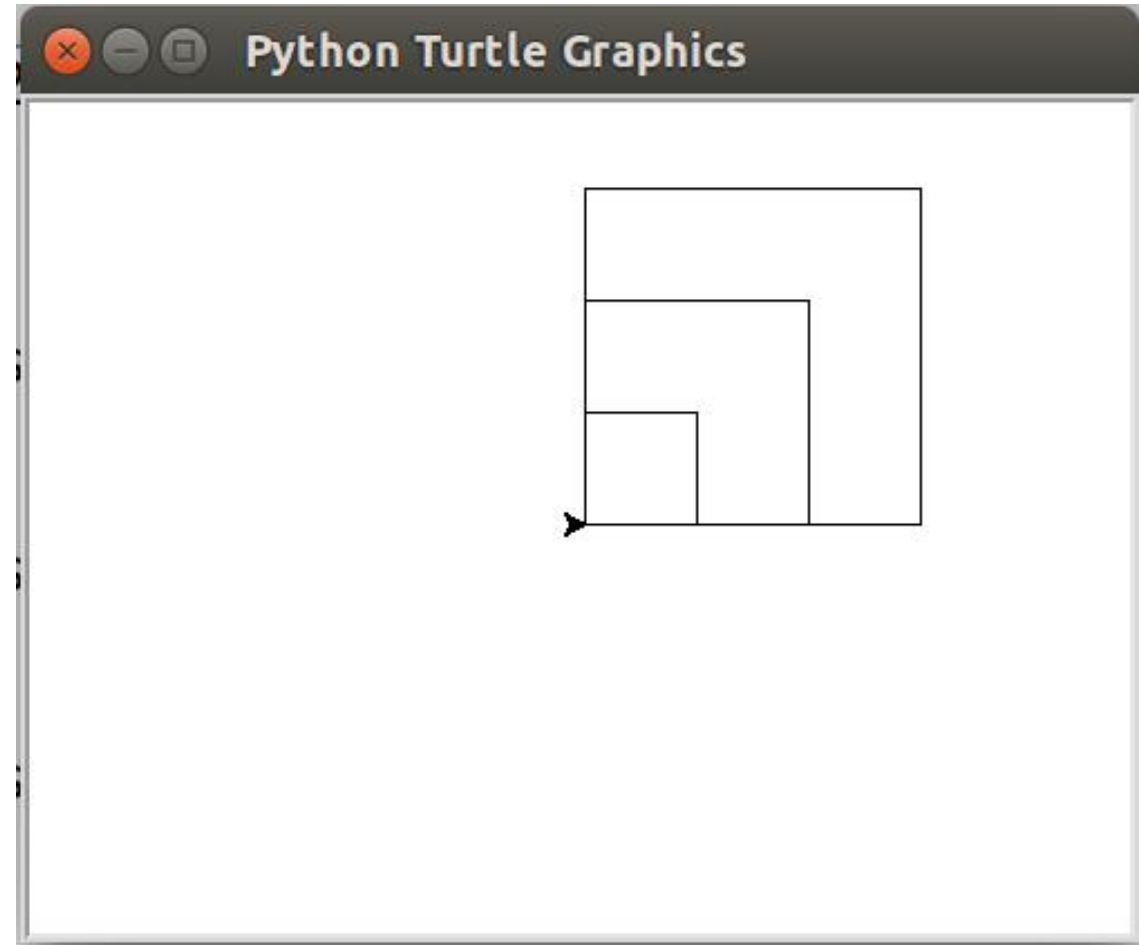
Using Iteration to Generate Squares

```
import turtle

def square(t, length):
    for s in range(4):
        t.forward(length)
        t.left(90)

t = turtle.getturtle()

square(t, 50)
square(t, 100)
square(t, 150)
```



- Convenient to define a Python function to draw a particular shape (**abstraction**).
- Example: square, the ``length" parameter is the length of the sides of the square.
- Call multiple times to draw squares of various sizes (**reuse**).

Controlling the Animation Speed of the Turtle

```
t = turtle.getturtle()  
t.speed(0)
```

- The animation speed is controlled by the turtle object's `speed()` method.
- Argument values 1 to 10 denote speed values of slowest (1) to almost fast (10)
- The **highest speed** is achieved with value 0.
- The **default speed** value is 3.

Controlling the Screen Refresh Rate

- While drawing, the screen is refreshed **periodically**.
- Refreshing the screen slows down the drawing process.
- The turtle module provides the `tracer()` function to **enable/disable screen updates**:

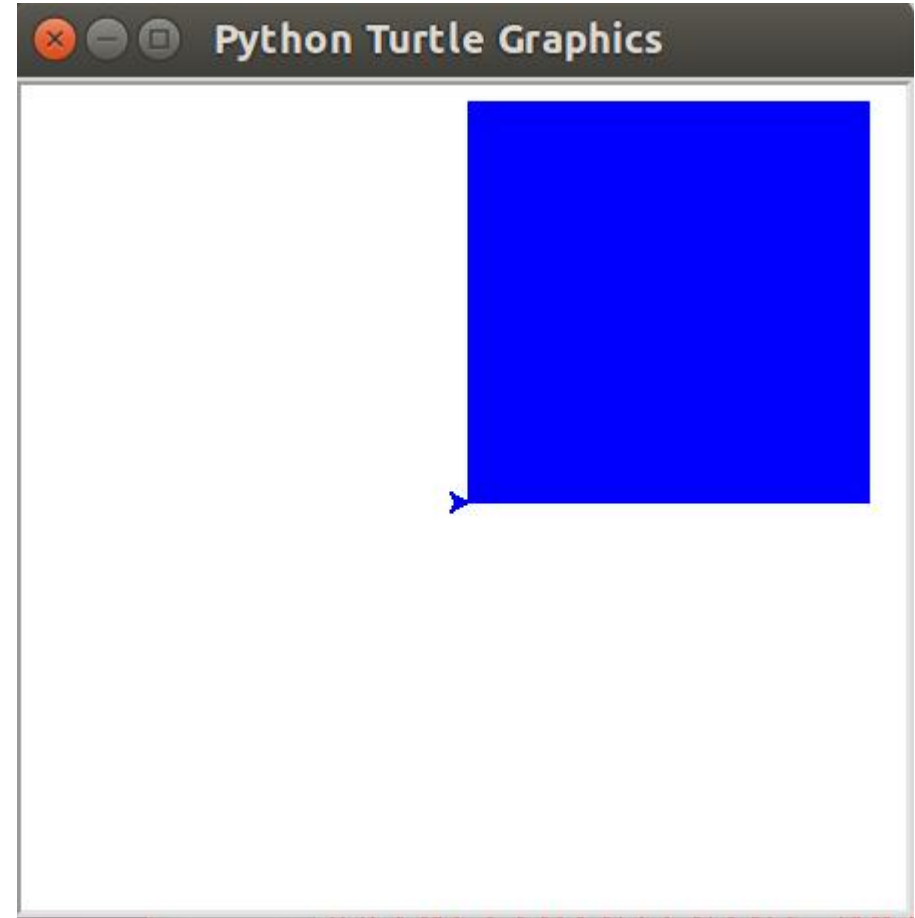
```
import turtle

turtle.tracer(0, 0) # disable screen updates
... # Draw here
turtle.update() # force 1 screen update
turtle.tracer(1, 10) # or restore default values for (periodic) updates
```

- To **disable screen updates**, call `turtle.tracer(0, 0)`
- After drawing has completed, call `turtle.update()` to **force a screen update**.
- Re-enable regular screen updates by calling `turtle.tracer(1, 10)`.
- **Note:** `tracer()` is a function provided by the turtle module.

Filling Shapes (1)

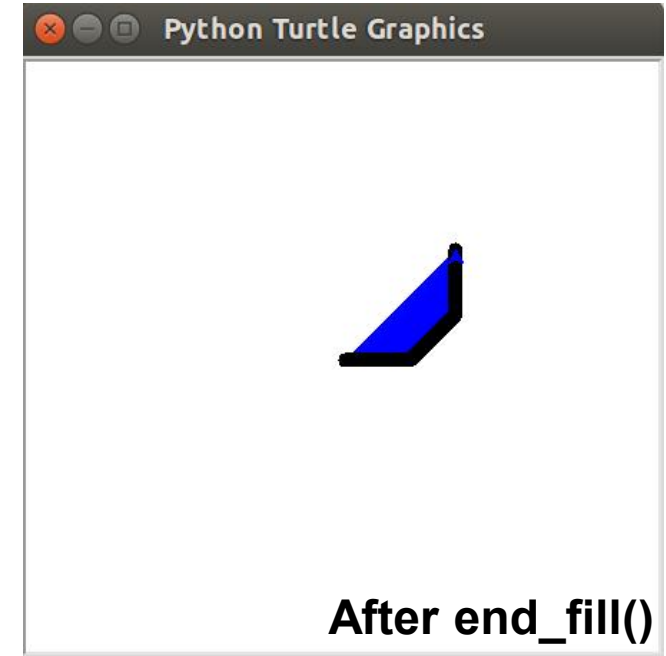
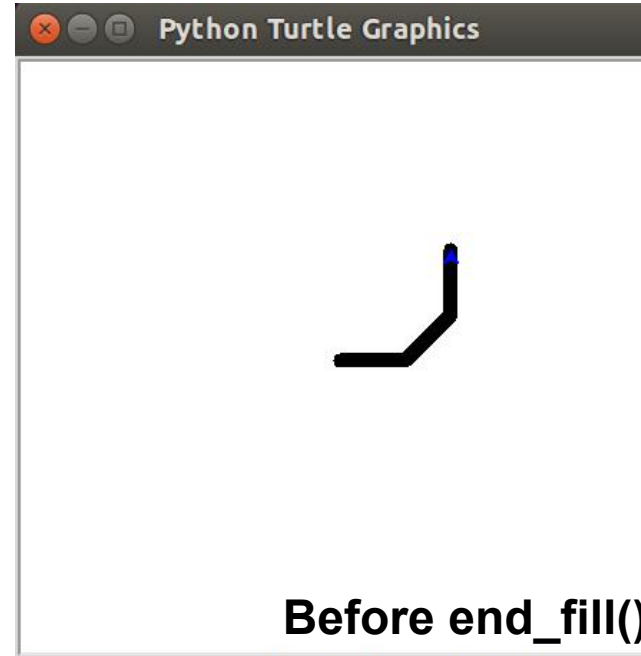
```
t.color("blue")  
t.begin_fill()  
square(t, 200)  
t.end_fill()
```



- To fill a shape, call the turtle's `begin_fill()` method.
- After drawing a shape, calling the turtle's `end_fill()` method will **fill** the shape with the turtle's current color.

Filling Shapes (2)

```
t.color("black")
t.width(10)
t.begin_fill()
t.forward(45)
t.setheading(45)
t.forward(45)
t.setheading(90)
t.forward(45)
t.color("blue")
input("Enter to continue: ")
t.end_fill()
```



- After drawing a shape, calling the turtle's `end_fill()` method will **fill** the shape.
- If the shape is not ``closed'', it will be closed by drawing a straight line from the current position to the start position of the shape.

Event-driven programming

- With **event-driven programming**, the control-flow of the program is driven by **events**.
 - User-initiated events: mouse clicks, key presses
 - Program-initiated events: expired timers
 - External events (not covered): sensor input, network requests, ...
- Main paradigm:
 - Connect a Python function to an event
 - When the event occurs, the function gets executed

Function calls versus references to functions (1)

```
>>> print(22)
```

```
22
```

```
>>> print
```

```
<built-in function print>
```

Function call to the print function

Reference to the print function object

- Typing the **name of a function** without parentheses and arguments refers to the **object** (code) of the **function**.
- Remember: names (like `print`) are references to objects in Python! 😊

```
>>> type(print)
```

```
<class 'builtin_function_or_method'>
```

```
>>> type(max)
```

```
<class 'builtin_function_or_method'>
```

```
>>> type(10)
```

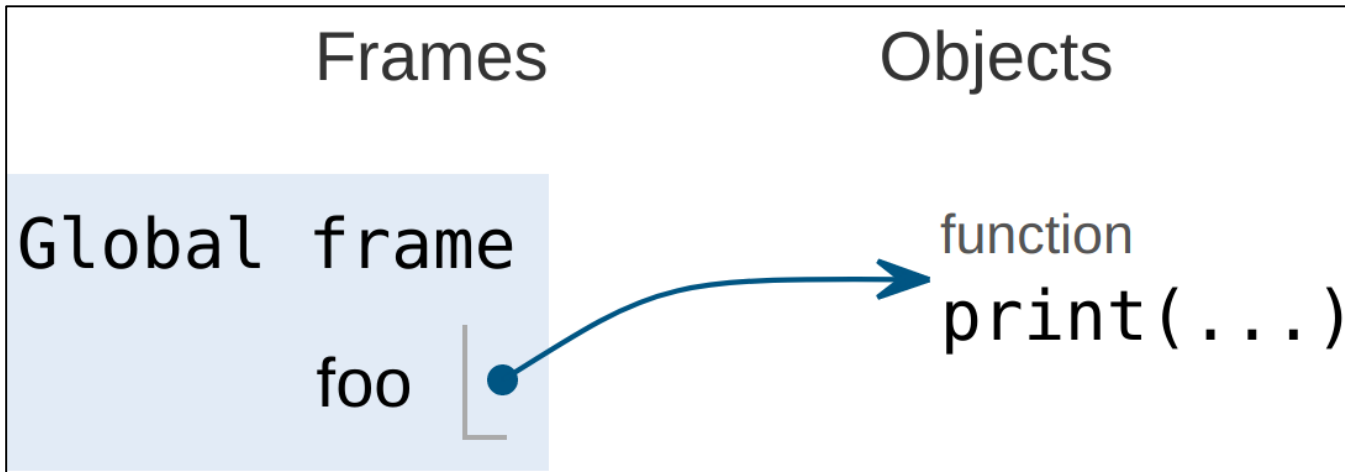
```
<class 'int'>
```

References to Functions (1)

We can assign print's reference to another variable:

```
>>> foo = print
>>> type(foo)
<class 'builtin_function_or_method'>
```

- The name `foo` is referring to the same function object as the name `print`.
- Names `foo` and `print` are aliases, `foo` is indistinguishable from `print`:



```
>>> id(foo)
1311008
>>> id(print)
1311008
```

same object id
→ same object

References to Functions (2)

- The name **foo** is referring to the same object as the name **print**.
- We can use **foo** as we would use **print**:

```
>>> foo = print
>>> foo('hello', 1+2)
hello 3
>>>
```

```
>>> print('hello', 1+2)
hello 3
>>>
```

References to Functions (3)

- Recap: references to functions enable us to **pass a function as an argument** to a higher-order function.

```
import math

def cco1100(f, x):
    """
    Higher-order function returning result of calling function f
    with argument x.
    """
    return f(x)

m = cco1100(abs, -2)
n = cco1100(math.factorial, 3)
o = cco1100(print, -2)
p = cco1100(input, 'Enter a number: ')
print(m, n, o, p)
```

Responding to User Mouse Clicks (1)

```
import turtle

turtle.setup(300, 200)

def printXY(x, y):
    print(x, y)

# Register callback for mouse on-click events:
turtle.onscreenclick(printXY)
turtle.mainloop()
```

*callback function
for mouse click events*

SimpleMouseCallbackFunction.py (LearnUs)

- To make our GUI respond to mouse clicks, we write a **function** that will be **called whenever the mouse is clicked on the graphics window**.
 - The name of such a function is **callback function**.
- We define a function that receives two arguments and prints them.

Responding to User Mouse Clicks (2)

```
import turtle

turtle.setup(300, 200)

def printXY(x, y):
    print(x, y)

# Register callback:
turtle.onscreenclick(printXY)

turtle.mainloop()
```

*register callback function
for mouse click events*

SimpleMouseCallbackFunction.py (LearnUs)

- The turtle module provides the `onscreenclick()` function to **register** a call-back function for mouse click events.
 - We provide the reference to our function `printXY` as argument.
 - Same mechanism as with the example on Slide 62.
- Python **expects** this callback function to **accept two arguments**.
 - Therefore the 2 parameters `x` and `y` with `printXY`.

Responding to User Mouse Clicks (3)

```
import turtle

turtle.setup(300, 200)

def printXY(x, y):
    print(x, y)

# Register callback for mouse on-click events:
turtle.onscreenclick(printXY)
turtle.mainloop()
```

*callback function
for mouse click events*

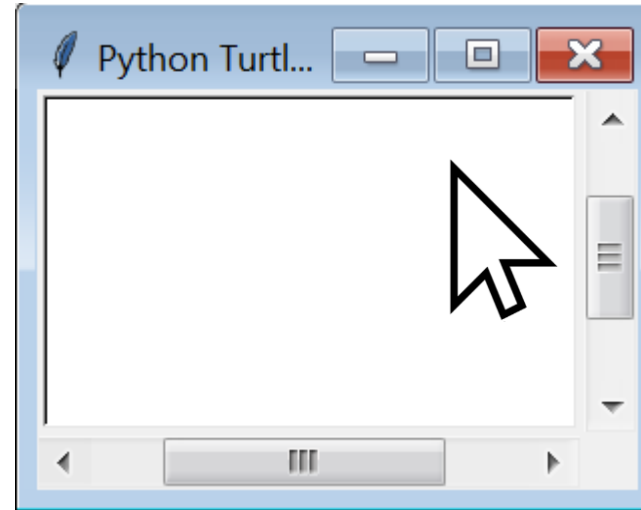
SimpleMouseCallbackFunction.py (LearnUs)

- Event processing is happening inside the `mainloop()` function.
 - Inside this function, Python keeps checking user inputs in an **infinite loop**.
 - Whenever a **mouse click is detected**, **Python calls our callback function `printXY`**.
 - With the call, Python provides the X and Y coordinates of the mouse cursor as arguments.
 - Our callback function prints the screen coordinates where the click event happened (**see following slide**).
 - The program does not terminate, because function `mainloop()` contains an infinite loop.

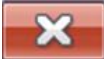
Responding to User Mouse Clicks (4)

```
83.0 49.0  
-108.0 51.0  
-108.0 51.0  
-71.0 46.0  
-69.0 1.0  
-11.0 26.0  
62.0 42.0  
64.0 2.0
```

Python Shell



Turtle window

- The blue output in PyCharm's shell is from our printXY() callback function.
- For every mouse click, the corresponding X and Y screen coordinates are printed.
- The program must be forcefully terminated by clicking the window's close button ().

Responding to User Key-press Events (1)

- Similarly to mouse clicks, we can register **callback functions** for particular **keys on the keyboard**.
- Whenever the user presses such a key, the corresponding callback function will be called.
- See example on following slides.

```
w = turtle.Screen()
t = turtle.Turtle()
```

```
def handler_1():
    t.forward(30)
```

```
def handler_2():
    t.left(45)
```

```
def handler_3():
    t.right(45)
```

```
def handler_4():
    w.bye() # Close the turtle window
```

Handler functions for

- moving the turtle forward
- turning left/right

"wire up" keypresses to our handlers we have defined:

```
w.onkey(handler_1, "Up")
w.onkey(handler_2, "Left")
w.onkey(handler_3, "Right")
w.onkey(handler_4, "q")
```

Tell the window to start listening for events.

```
w.listen()
w.mainloop()
```

```
w = turtle.Screen()
t = turtle.Turtle()
```

```
def handler_1():
    t.forward(30)
```

```
def handler_2():
    t.left(45)
```

```
def handler_3():
    t.right(45)
```

```
def handler_4():
    w.bye()
```

■ Handler function to close the turtle window.
■ Closing the turtle window will terminate the loop inside mainloop().

"wire up" keypresses to our handlers we have defined:

```
w.onkey(handler_1, "Up")
w.onkey(handler_2, "Left")
w.onkey(handler_3, "Right")
w.onkey(handler_4, "q")
```

Tell the window to start listening for events.

```
w.listen()
w.mainloop()
```

```
w = turtle.Screen()
t = turtle.Turtle()
```

```
def handler_1():
    t.forward(30)
```

```
def handler_2():
    t.left(45)
```

```
def handler_3():
    t.right(45)
```

```
def handler_4():
    w.bye()
```

"wire up" keypresses to our handlers we have defined:

```
w.onkey(handler_1, "Up")
w.onkey(handler_2, "Left")
w.onkey(handler_3, "Right")
w.onkey(handler_4, "q")
```



Assign handler functions to keyboard events.

- "Up" → handler_1
- "Left" → handler_2
- "Right" → handler_3
- "q" → handler_4

```
w.listen()
w.mainloop()
```

```
w = turtle.Screen()
t = turtle.Turtle()
```

```
def handler_1():
    t.forward(30)
```

```
def handler_2():
    t.left(45)
```

```
def handler_3():
    t.right(45)
```

```
def handler_4():
    w.bye()
```

```
# "wire up" keypresses:
```

```
w.onkey(handler_1, "Up")
w.onkey(handler_2, "Left")
w.onkey(handler_3, "Right")
w.onkey(handler_4, "q")
```

```
# Start listening for events:
```

```
w.listen()
w.mainloop()
```

- `listen()` method sets the **focus** to the window.
 - Required but not well-documented. 😞
- Calling `mainloop()` starts event processing.

```
w = turtle.Screen()
```

```
t = turtle.Turtle()
```

```
def handler_1():
```

```
    t.forward(30)
```

Cannot provide turtle t as
additional argument here!



```
w.onkey(handler_1, "Up")
```

- Callback functions are one example where the use of **global variables** is **indispensable**.
- The handler_1 function needs the turtle t to call the forward() method.
- The onkey() method does not allow us to provide the turtle t as an additional argument
 - **Wrong:** w.onkey(handler_1(t), "Up")
- **But:** the general sentiment against global variables still holds (you shall avoid global variables wherever possible).

Responding to Timer Events

- Similarly to events from the user, we can set up a callback function in response to a timer event.
- The **time** after which the callback function shall be called is specified in milliseconds (ms) to function **ontimer()**.
- **Note:** timer is **one-shot**, ontimer() returns immediately.

```
w = turtle.Screen()
t = turtle.Turtle()

def T_Handler():
    t.left(45)
    t.forward(50)      # move turtle
    w.ontimer(T_Handler, 250) # w will call T_Handler in 250 ms from now
    return # ontimer() returns immediately

T_Handler()
w.mainloop()
```

Event-driven Programming (Wrap-up)

- The programming style from the previous slides is called **event-driven programming**.
- The control-flow of the program is driven by **events**.
 - user actions (mouse clicks, key-presses)
 - timers
 - sensor input, network requests, ...
- With event-driven programming, the program is running in an ``infinite'' main loop which checks for events.
- For each detected event with a registered callback function, the callback function will be called.
- This programming style is predominant with graphical user interfaces (GUIs) and computer games.